

PROGRAMMING LANGUAGES

LAB INTRODUCTION

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026



© Michele Pasqua, 2026

LECTURER INFO



Who I AM

Michele Pasqua, PhD – *Professor at the Dept. of Computer Science*

 <https://michelepasqua.github.io>

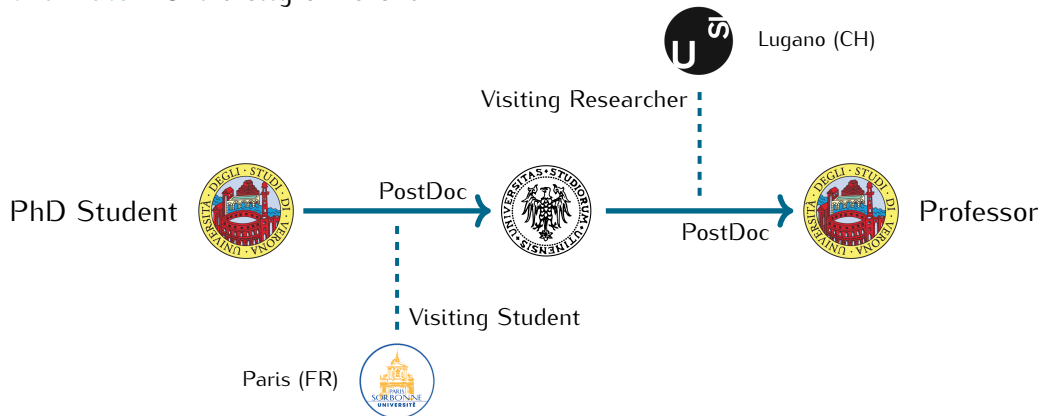
Main **research interests**:

- ▶ Program verification and validation
- ▶ Programming language design and implementation
- ▶ Code protection and software security



Who I AM (CONT'D)

Alma mater: University of Verona



WHAT I DO

Co-founder of the University of Verona Software Engineering Lab



 <https://universe-lab.pages.dev>



Prof. Mariano Ceccato, PhD – *Lab head*

WHAT I DO (CONT'D)



Co-founder of KonTrust Srl

UniVR spin-off company recently born

- ▶ Real-time automated tracking of IoT data
- ▶ Blockchain-based non-repudiation
- ▶ Anti-tamper infrastructure



WHAT I DO (CONT'D)

Italian training program on [cybersecurity](#) – *Verona team leader*



- ▶ Lectures and labs on cybersecurity
- ▶ Competition among selected Italian universities
- ▶ Step-up to European level (hopefully)



CONTACTS

How to reach me out:

-  Knock at my office door @ Cà Vignal 2, Floor 2, Room 24
-  Catch me after a class
-  Send me an email @ michele.pasqua@univr.it

To ask about the course, internships, theses, and research projects!

COURSE ORGANIZATION



COURSE DETAILS

Lab module (12⁺ hours) 

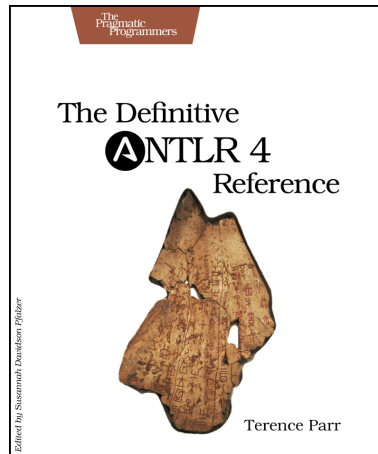
- ▶ Sessions on Wednesdays (09:30) – *2 hours at Aula Delta*

Office hours by appointment only (via email)

COURSE MATERIAL

☰ The Definitive ANTLR 4 Reference
by *Terence Parr*

The Pragmatic Programmers, 2/e – 2013



COURSE MATERIAL (CONT'D)

Available on Moodle:

- ▶ Course slides
- ▶ Lab exercises and solutions
- ▶ Course announcements
- ▶ Pointers to advanced topics



 <https://moodledidattica.univr.it/course/view.php?id=21942>

COURSE EXAM AND GRADE

The exam will consist in developing an interpreter for a language of your design

- ▶ Implementation can be in [Java](#) or any other language supported by ANTLR¹
- ▶ The language must satisfy some minimal functional requirements
- ▶ The language must be documented into a short report

The final grade will be given after an oral discussion of the developed language

¹Except JavaScript/TypeScript for obvious reasons ...

PROGRAMS AND INTERPRETERS



INTERPRETER (INFORMAL)

We aim at building a **magic box** (an interpreter)



- ▶ That given a program P , written in some programming language
- ▶ Is able to execute P on any given input i
- ▶ ...we want to build it as much **automatically** as possible

FORMAL LANGUAGES

A **formal language** is a set of finite strings over a finite set of symbols

- ▶ Σ is finite set of symbols (e.g., UTF-8)
- ▶ $\mathcal{L} \subseteq \Sigma^*$ is a set of finite strings (e.g., all Java programs)

$$\Sigma^* = \bigcup_{n \in \mathbb{N}} \Sigma^n \quad \Sigma^n = \begin{cases} \{\varepsilon\} & \text{if } n = 0 \\ \{ws \mid w \in \Sigma^{n-1} \wedge s \in \Sigma\} & \text{if } n > 0 \end{cases}$$

A **programming language** is a formal language

- ▶ Having a **semantics** for its elements
- ▶ Elements $P \in \mathcal{L}$ are called **programs**

PROGRAM SEMANTICS

Programs are just strings, we need to attach a **meaning** to them

- ▶ Typically a function computing something

Given a program $P \in \mathcal{L}$, its meaning is $\mathcal{L}[[P]] : \text{In} \rightarrow \text{Out}$

- ▶ $\mathcal{L}[[P]](i) = o$ means that P transforms input i into output o
- ▶ $\mathcal{L}[[\cdot]]$ is the **semantics** of $\mathcal{L}$

PROGRAM SEMANTICS (CONT'D)

Let $\mathcal{L}$ be Java and P the following Java class

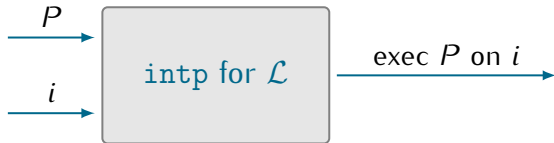
```
public class MainFactorial {  
  
    public static void main(String[] args) {  
        System.out.println(factorial(Integer.parseInt(args[0])));  
    }  
  
    private static int factorial(int n) {  
        if (n <= 0) return 1;  
        else return n * factorial(n - 1);  
    }  
}
```

Then, $\text{Java}[\![\text{MainFactorial}]\!](5) = 120$

- ▶ With `In` being the command-line arguments
- ▶ And `Out` being the standard output

INTERPRETER (MORE FORMAL)

We aim at building an **interpreter** for a language $\mathcal{L}$

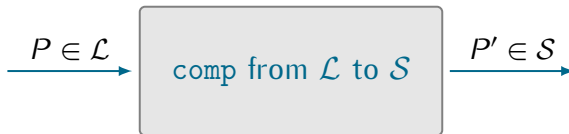


Such that $^S[\text{intp}](P, i) = ^{\mathcal{L}}[P](i)$, for any $P \in \mathcal{L}$ and $i \in \text{In}$

- ▶ The interpreter does not need to be written in $\mathcal{L}$
- ▶ Oracle JVM is an interpreter for Java programs, written in C++

COMPILER

A **compiler** translates programs from a language to another

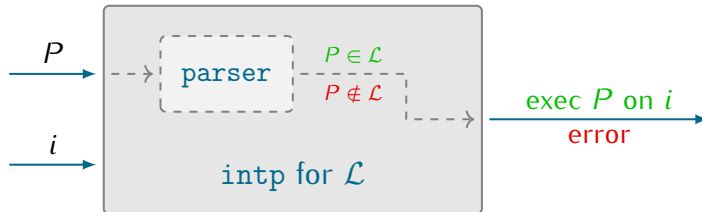


Such that $\mathcal{L} \llbracket P \rrbracket = \mathcal{S} \llbracket^{\mathcal{R}} \llbracket \text{comp} \rrbracket (P) \rrbracket = \mathcal{S} \llbracket P' \rrbracket$, for any $P \in \mathcal{L}$

- ▶ The compiler does not need to be written in either $\mathcal{L}$ or $\mathcal{S}$
- ▶ GNU GCC is a compiler from Go programs to binary code, written in C++

PARSING

We aim at building an **interpreter** for a language $\mathcal{L}$



A parser for $\mathcal{L}$ implements a function $s_{\llbracket \text{parser} \rrbracket} : \Sigma^* \rightarrow \{yes, no\}$ such that

$$s_{\llbracket \text{parser} \rrbracket}(w) = \begin{cases} yes & \text{if } w \in \mathcal{L} \\ no & \text{if } w \notin \mathcal{L} \end{cases} \quad \text{for any } w \in \Sigma^*$$

PARSING PROBLEMS

Checking if $w \in \mathcal{L}$ is **undecidable** for unconstrained languages 🚫

- ▶ It reduces to the halting problem

Checking if $w \in \mathcal{L}$ is **decidable** for context-free languages 😊

- ▶ That is, $\mathcal{L}$ is generated by a context-free grammar
- ▶ Productions of the form: $A \rightarrow \alpha$

Can we have not context-free programming languages?

- ▶ In principles yes
- ▶ Not in practice (no compilers, no interpreters)

CONTEXT-FREE GRAMMARS

Binary words language $\{w \in \Sigma^* \mid w \in \{0, 1\}^*\}$

// *regular, right-linear*

$$\begin{aligned} S &\rightarrow 0S \mid 1S \\ S &\rightarrow \epsilon \end{aligned}$$

Binary palindromes language $\{w \in \{0, 1\}^* \mid w = \overleftarrow{w}\}$

// *context-free*

$$\begin{aligned} S &\rightarrow 0S0 \mid 1S1 \\ S &\rightarrow 0 \mid 1 \mid \epsilon \end{aligned}$$

Binary copy language $\{ww \in \Sigma^* \mid w \in \{0, 1\}^*\}$

// *context-sensitive*

S	$\rightarrow$	$0AS \mid 1BS \mid C$	C	$\rightarrow$	ϵ
$A0$	$\rightarrow$	$0A$	$A1$	$\rightarrow$	$1A$
$B0$	$\rightarrow$	$0B$	$B1$	$\rightarrow$	$1B$
AC	$\rightarrow$	$C0$	BC	$\rightarrow$	$C1$

AUTOMATED PARSER GENERATION



ANTLR

ANother Tool for Language Recognition

- ▶ Essentially, a parser generator

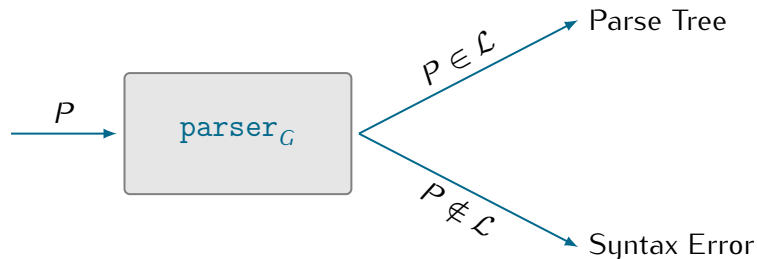


Such that $\mathcal{L}$ is the language generated by G

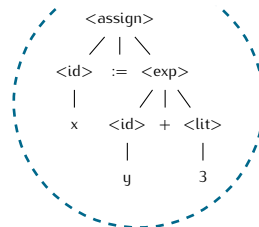
ANTLR (CONT'D)

ANother Tool for Language Recognition

- Essentially, a parser generator



Define semantics



REQUIREMENTS

First, one needs to install the `jdk` *// usually, the `jre` comes with the `jdk`*

```
apt install default-jdk or apt install openjdk-21-jdk
```

Then, one just needs a text editor to program *// like `gedit`, `vim` or `emacs`*

From <https://www.antlr.org/download.html>

- ▶ Download the ANTLR v4 **complete** binaries, to generate the parser classes
- ▶ Download the the ANTLR v4 **runtime** binaries, to compile and use the parser

INTEGRATED DEVELOPMENT ENVIRONMENT

An IDE is a toolchain to support programmers

Available IDEs for Java

- ▶ IntelliJ IDEA <https://www.jetbrains.com/idea/download>
- ▶ Eclipse <https://www.eclipse.org/downloads>
- ▶ NetBeans <https://netbeans.apache.org/front/main/download>
- ▶ Visual Studio Code <https://code.visualstudio.com/docs/languages/java>

INTEGRATED DEVELOPMENT ENVIRONMENT (CONT'D)

Advantages of an IDE

- ▶ Provide syntax highlighting
- ▶ Provide live code linting *// type checking, code smells, etc.*
- ▶ Add pretty printing support
- ▶ Simplify code running and test *// integrated debugger*
- ▶ Provide graphical project organization and navigation
- ▶ Integrate version control tools *// git*
- ▶ Integrate build tools *// for Java: maven, gradle or ant*

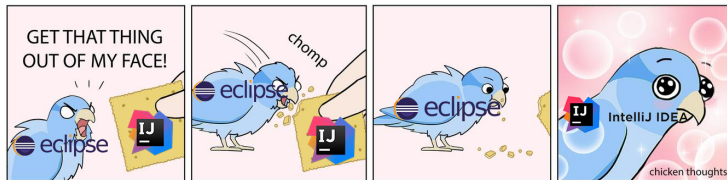
 Integrate a nice ANTLR v4 plugin

Which IDE?



Which IDE? (CONT'D)

You can choose the IDE you prefer, in the lab we will use IntelliJ IDEA



It is a proprietary software, but

- ▶ The Community Edition is free to use
- ▶ The Ultimate Edition is still free for university students

SETUP

Install the ANTLR v4 plugin

// e.g., for IntelliJ IDEA

- ▶ Got to File -> Settings -> Plugins
- ▶ Search “antlr”
- ▶ Select ANTLR v4

Create a BinWords.g4 grammar file into the project source

- ▶ Implement the grammar for the language $\{w \in \Sigma \mid w \in \{0,1\}^*\}$
- ▶ Run the ANTLR v4 plugin to parse an input string on-the-fly

SETUP (CONT'D)

Add the ANTLR v4 runtime to the project

// e.g., for Maven

```
<dependencies>          <!-- to add to the pom.xml Maven configuration file -->
  <dependency>
    <groupId>org.antlr</groupId>
    <artifactId>antlr4-runtime</artifactId>
    <version>4.13.2</version>
  </dependency>
</dependencies>
```

Right-click the BinWords.g4 and select Generate ANTLR recognizer

- This will create a parser for the binary words language

NOT ONLY INTERPRETATION

ANTLR is a versatile tool

- ▶ Analysis of complex structured files (e.g., XML configurations)
- ▶ Automated code migration
- ▶ Query languages or communication protocols
- ▶ Structural code analysis (linter)

FURTHER READING

The website <https://www.antlr.org> provides examples, documentation, tutorials

- ▶ There is also an online playground <http://lab.antlr.org>
- ▶ And the book about ANTLR

✂ Implement the CF grammar for the language of binary palindromes

✂ Implement a CF grammar for the language of even binary palindromes:

$$\{w \in \{0, 1\}^* \mid w = \overleftarrow{w} \wedge |w| \text{ is even}\}$$

✂ Implement a CF grammar for the language of odd binary palindromes:

$$\{w \in \{0, 1\}^* \mid w = \overleftarrow{w} \wedge |w| \text{ is odd}\}$$

Q + A



© Michele Pasqua, 2026

This work is licensed under the  **Creative Commons** Attribution-NonCommercial-NoDerivatives 4.0 International license. To view a copy of the license, visit:

<http://creativecommons.org/licenses/by-nc-nd/4.0>

You are free to:

 **Share** – Copy, distribute, and display the work

Under the following terms:

 **Attribution** – You must attribute the work in the manner specified by the author or licensor

 **NonCommercial** – You may not use this work for commercial purposes

 **NoDerivatives** – If you alter, transform, or build upon this work, you may not distribute the modified material

For any reuse or distribution, you must make clear to others the license terms of this work. Any of these conditions can be waived if you get permission from the copyright holder. Your fair use and other rights are in no way affected by the above.